Distributed Money Transfer System – Final Project

1. Overview

In this project, you will implement a simplified version of a blockchain-inspired money transfer system in Java. Instead of miners and proof-of-work, the "approval" mechanism is based on the social graph of past transactions: when a user wants to send money to another user, the system finds the shortest path between them in the graph of past transactions. The intermediate users on that path, not just those who transacted directly with the sender, become the approvers of the transfer.

The system is not a real blockchain. It is a simulation that borrows the core idea of "distributed approval" and adapts it to a graph-based model. Your job is to build the data structures, the algorithms (heap sort, Breadth-First Search (BFS), hashing), the approval logic, and the logging mechanism.

2. Learning Outcomes

By completing this project, you will:

- Implement and use custom data structures (singly linked list, queue) without relying on Java's built-in collections.
- Apply heap sort to order pending transactions by sender priority.
- Implement BFS over a graph derived from historical transactions.
- Build a custom hash index with a fixed hash function (id % 4).
- Write structured log output to a file for auditing.

3. System Components

3.1 User

Represents a registered user. Fields:

- *id* : unique identifier
- *name* : display name
- *balance* : current balance
- *priority* : importance value; higher means more important

The priority is used later to order pending transactions.

3.2 Transaction

Represents a money transfer. Fields:

- *senderId, receiverId, amount*
- *createdAt, approvedAt (timestamps)*
- *approvers*: list of intermediate users who approved
- *approvals*: map of approver id to decision (true/false)

A Transaction object is reused across the whole lifecycle: it starts in the pending queue, moves through BFS and approval, and either ends up in the confirmed list or in the log file as cancelled.

3.3 HashIndex

A custom hash table built on id, using the hash function id % 4. Each bucket is a chain of nodes; each node corresponds to one user and holds references to all transactions that user participated in (as sender or receiver).

The hash index is used by BFS to quickly find a user's neighbors (users they have transacted with in the past).

3.4 HeapSort

A custom heap sort implementation that orders a list of transactions by the priority of their sender. Higher priority goes to the front of the queue.

3.5 BFS

A BFS that finds the shortest path between the sender and the receiver in the transaction graph. The intermediate users on that path become the approvers of the transaction.

Important rules:

- The direct edge between sender and receiver is ignored. Even if they have transacted before, BFS must find another route through at least one intermediate user.
- The graph is undirected: if A and B have transacted, they are neighbors of each other.

3.6 MyLinkedList (custom singly linked list)

A custom implementation of a singly linked list. It is used for the confirmed transactions list. **You can not use java.util.LinkedList or any other built-in collection inside this class.**

3.7 MyQueue (custom queue)

A custom implementation of a queue. It is used for:

- the pending transactions queue (transactions waiting to be processed)
- the BFS to find the approvers

You can not use java.util.Queue, ArrayDeque, LinkedList, or any other built-in collection inside this class.

3.8 Logger

Writes all transaction processing events to transaction_log.log. On startup, the log file is deleted and recreated so each run starts clean.

3.9 SystemState

Holds all shared data:

- *users*: map of user id to User
- *confirmedTransactions*: List of approved transactions
- *hashIndex*: the hash index
- *pendingQueue*: Queue of pending transactions

4. Overall Flow

The following steps describe how a single transaction travels through the system.

Step 1: User submits a transfer
The user chooses menu option "Send Money" and *enters sender id, receiver id,* and *amount.* The system validates the input:

- sender and receiver must be different
- both users must exist
- amount must be positive

Step 2: Transaction is enqueued and sorted
A new Transaction is created and added to the pending queue. After every insertion, the entire queue is re-sorted using **heap sort**, based on the sender's priority. The highest-priority transaction is always at the front.

Step 3: Scheduler picks one transaction every 10 seconds
A background scheduler fires every 10 seconds and takes exactly one transaction from the front of the queue. This transaction is handed off to a worker thread so the menu stays responsive. The worker thread processes the transaction and then terminates.

Step 4: BFS finds the approvers
Inside the worker thread:

- BFS starts from the sender.
- It uses the hash index to retrieve each user's neighbors.
- It ignores the direct sender–receiver edge.
- When the receiver is reached, the path is reconstructed.
- The intermediate users on the path become the approvers.

If no path exists, the transaction is cancelled and logged with reason "no path".

Step 5: Approvers vote
Each approver checks whether the sender's current balance is at least the transaction amount. If yes, the approver votes APPROVED; otherwise REJECTED. Each decision is stored in the transaction's approvals map.

Step 6: Final decision
If all approvers approved, the transaction is committed:

- approvedAt is stamped with the current time
- the transaction is appended to the confirmed linked list
- the transaction is added to the hash index
- sender balance decreases, receiver balance increases

Otherwise, the transaction is cancelled and logged with reason "insufficient balance or rejected by approver".

5. Key Rules and Constraints

- No built-in collections can be used inside MyLinkedList and MyQueue. Only your own node objects are allowed.
- Hash function is fixed: id % 4.
- BFS must ignore the direct edge between sender and receiver.
- Graph is undirected.
- Sender and receiver cannot be the same user.
- Approvers are only the intermediate users on the shortest path, not the sender or the receiver.
- Balance is checked at processing time, not at enqueue time.
- Balance is updated only after successful commit.
- Every cancellation must be written to the log file with a reason.

6. Threading Model (Provided in the Template)

The threading model is already implemented in the template code that you will receive. You do not need to write it yourself. Your job is to make sure your data structures and algorithms work correctly inside this model.

The template provides:

- A *ScheduledExecutorService* that fires every 10 seconds and dequeues exactly one pending transaction.
- An *ExecutorService* that runs each dequeued transaction on its own worker thread, so the worker dies once the transaction is processed.
- A menu loop running on the main thread, fully independent of the scheduler and worker threads.
- synchronized methods on SystemState to protect the shared structures (queue, linked list, hash index, user map).

7. Logging

The *Logger* class is already implemented and provided inside the template project, so you do not need to write it yourself. A sample transaction_log.log file is also included in the template project so you can see the expected format.

All processing events are written to transaction_log.log. The log file is deleted and recreated on every program start, so each run produces a clean log.

Logged events include:

- A new transaction being enqueued
- A transaction starting to be processed
- The list of approvers chosen by BFS
- Each approver's decision (APPROVED / REJECTED)
- Confirmation of a successful transaction
- Cancellation of a transaction with the reason

8. Sample Data and Expected Behaviors

This section contains the 10 sample users, the 10 past transactions, and the neighbor relationships that arise from these transactions. It then simulates how 3 new transactions would be resolved.

id	**name**	**balance**	**priority**
1	Ali	1000	5
2	Ayşe	1500	3
3	Mehmet	800	7
4	Zeynep	2000	2
5	Can	1200	8
6	Elif	900	4
7	Burak	900	6
8	Deniz	600	9
9	Ece	1300	1
10	Furkan	1100	10

Table 1 10 Sample Users

# of transaction	Sender	Receiver	Amount
1	1	2	100
2	2	3	200
3	3	4	150
4	2	4	50
5	4	5	300
6	5	6	80
7	7	8	110
8	8	9	90
9	9	10	60
10	8	10	40

Table 2 10 past transactions (sender, receiver, amount)

id	name	Neighbors
1	Ali	2
2	Ayse	1, 3, 4
3	Mehmet	2, 4
4	Zeynep	2, 3, 5
5	Can	4, 6
6	Elif	5
7	Burak	8
8	Deniz	7, 9, 10
9	Ece	8, 10
10	Furkan	8, 9

Table 3 Neighbors of each user (undirected graph)

Assume there are three transactions in the pending queue, ordered by sender priority:

1. sender = 3 (Mehmet), receiver = 4 (Zeynep), amount = 5000
2. sender = 1 (Ali), receiver = 6 (Elif), amount = 100
3. sender = 2 (Ayse), receiver = 9 (Ece), amount = 150

Transaction 1: Path exists, balance insufficient -> CANCELLED

sender = 3 (Mehmet), receiver = 4 (Zeynep), amount = 5000

- Mehmet's balance is 800, which is < 5000.
- BFS finds a path from 3 to 4. Note that the direct edge 3 ↔ 4 exists in the past transactions, but BFS must ignore it, so it finds another route such as 3 -> 2 -> 4.
- The intermediate user (2) becomes the approver.
- The approver checks Mehmet's balance: 800 < 5000 → REJECTED.
- Not all approvers approved -> the transaction is cancelled.
- Log entry: [CANCELLED] ... reason=insufficient balance or rejected by approver.

Transaction 2: Path exists, balance sufficient -> APPROVED

sender = 1 (Ali), receiver = 6 (Elif), amount = 100

- Ali's balance is 1000, which is >= 100.
- BFS finds a path from 1 to 6, e.g. 1 -> 2 -> 4 -> 5 -> 6.
- The intermediate users (2, 4, 5) become approvers.
- Each approver checks Ali's balance: 1000 >= 100: APPROVED.
- All approvers approve -> the transaction is committed.
- Balances update: Ali 900, Elif 1000.

The transaction is added to the confirmed list and to the hash index. This means the graph is updated: a new edge is created between Ali (1) and Elif (6).

id	name	Neighbors
1	Ali	2,6
6	Elif	5,1

Transaction 3: No path -> CANCELLED

sender = 2 (Ayse), receiver = 9 (Ece), amount = 150

- There is no edge between the two users, so BFS exhausts the queue without reaching Ece.
- The transaction is cancelled.
- Log entry: [CANCELLED] ... reason=no path.

9. Console Menu

When the program starts, it presents an interactive menu on the main thread. While this menu is open, the scheduler and worker threads continue running in the background, so transactions are processed every 10 seconds even if the user is not interacting with the menu**.**

The menu offers the following options:

1. **Send Money**
   Prompts the user to enter a sender id, a receiver id, and an amount. The input is validated (sender and receiver must be different, both users must exist, amount must be positive). If the input is valid, a new Transaction is created and added to the pending queue. The new transaction will be processed automatically within the next 10-second cycles, depending on its priority.

2. **View Pending Transactions**
   Displays all transactions currently waiting in the pending queue, in priority order (highest priority first). For each transaction, it shows the sender id, sender name, sender priority, receiver id, amount, and createdAt timestamp. This option is useful to see which transaction will be processed next and to verify that heap sort is working correctly.

```
========= MENU =========
1) Send Money
2) View Pending Transactions
3) View Confirmed Transactions
4) View Users
5) Exit
Choice: 2

--- PENDING TRANSACTIONS (priority order) ---
sender=3(Mehmet, pri=7) -> receiver=4 amount=5000,00 createdAt=Thu Oct 08 22:47:08 TRT 2026
sender=1(Ali, pri=5) -> receiver=6 amount=100,00 createdAt=Thu Oct 08 22:47:08 TRT 2026
sender=2(Ayse, pri=3) -> receiver=9 amount=150,00 createdAt=Thu Oct 08 22:47:08 TRT 2026
```

Figure 1 Example view of the pending queue

3. **View Confirmed Transactions**
   Displays all transactions that have been successfully approved and committed, in insertion order (oldest first). Each entry shows the full transaction details, including sender, receiver, amount, createdAt, approvedAt, the list of approvers, and each approver's decision. This is the permanent history of all completed transfers.

```
========= MENU =========
1) Send Money
2) View Pending Transactions
3) View Confirmed Transactions
4) View Users
5) Exit
Choice: 3

--- CONFIRMED TRANSACTIONS ---
Txn[sender=1, receiver=2, amount=100,00, createdAt=2026-10-08 22:47:08, approvedAt=2026-10-08 22:47:08, approvers=[
Txn[sender=2, receiver=3, amount=200,00, createdAt=2026-10-08 22:47:08, approvedAt=2026-10-08 22:47:08, approvers=[
Txn[sender=3, receiver=4, amount=150,00, createdAt=2026-10-08 22:47:08, approvedAt=2026-10-08 22:47:08, approvers=[
Txn[sender=2, receiver=4, amount=50,00, createdAt=2026-10-08 22:47:08, approvedAt=2026-10-08 22:47:08, approvers=[]
Txn[sender=4, receiver=5, amount=300,00, createdAt=2026-10-08 22:47:08, approvedAt=2026-10-08 22:47:08, approvers=[
Txn[sender=5, receiver=6, amount=80,00, createdAt=2026-10-08 22:47:08, approvedAt=2026-10-08 22:47:08, approvers=[]
Txn[sender=7, receiver=8, amount=110,00, createdAt=2026-10-08 22:47:08, approvedAt=2026-10-08 22:47:08, approvers=[
Txn[sender=8, receiver=9, amount=90,00, createdAt=2026-10-08 22:47:08, approvedAt=2026-10-08 22:47:08, approvers=[]
Txn[sender=9, receiver=10, amount=60,00, createdAt=2026-10-08 22:47:08, approvedAt=2026-10-08 22:47:08, approvers=[
Txn[sender=8, receiver=10, amount=40,00, createdAt=2026-10-08 22:47:08, approvedAt=2026-10-08 22:47:08, approvers=[
Txn[sender=1, receiver=6, amount=100,00, createdAt=2026-10-08 22:47:08, approvedAt=2026-10-08 22:47:28, approvers=[
```

Figure 2 Example view of the confirmed list

4. **View Users**
   Displays all registered users with their current balance and priority. After transactions are

committed, balances change, so this option lets the user verify that money was correctly transferred from sender to receiver.

```
--- USERS ---
User{id=1, name='Ali', balance=900,00, priority=5}
User{id=2, name='Ayse', balance=1500,00, priority=3}
User{id=3, name='Mehmet', balance=800,00, priority=7}
User{id=4, name='Zeynep', balance=2000,00, priority=2}
User{id=5, name='Can', balance=1200,00, priority=8}
User{id=6, name='Elif', balance=1000,00, priority=4}
User{id=7, name='Burak', balance=900,00, priority=6}
User{id=8, name='Deniz', balance=600,00, priority=9}
User{id=9, name='Ece', balance=1300,00, priority=1}
User{id=10, name='Furkan', balance=1100,00, priority=10}

========= MENU =========
1) Send Money
2) View Pending Transactions
3) View Confirmed Transactions
4) View Users
5) Exit
Choice:
```

Figure 3 Example view of the users list

5. **Exit**
   Stops the scheduler and the worker pool, and terminates the program cleanly. Any pending transactions that have not yet been processed are discarded.

10. Strict Rules

Read this section carefully. Violating any of the rules below will result in your project being rejected.

10.1 No Built-in Collection Classes

You must not use any of the following predefined Java classes: LinkedList, Queue, Deque, ArrayDeque, Stack.

If any of these appear in your code, **your project will be rejected** regardless of whether it works.

You **must** use your own implementations of **MyLinkedList, Hash Index** and **MyQueue**.

Do not use ArrayList as a replacement for MyLinkedList or MyQueue.

10.2 Do Not Modify Existing Class or Method Signatures

The template provides:

- The class names.
- The method names and their signatures (parameters and return types).
- The class fields.

You **must not:**

- Rename any class, method, or field.
- Change any method signature.
- Add new classes.
- Add new methods.

You **can:**

- Fill in the bodies of the existing methods (the ones marked TODO).
- Declare local variables inside methods.

Example: You may write int distance = ...; inside a method. You can not write a new method like private int calculateDistance(...).

11. Submission

- **Deadline:** The final submission deadline is **December 20, 2026, at 23:59**.
- **What to submit:** You must fill in all the classes and methods provided in the template project, and submit the **entire project folder** as a single .zip file.
- **File naming:** The zip file must be named using the following pattern: studentID_ProjectType.zip

  where ProjectType is either SnakeGame or DistributedMoneyTransferSystem.
- **Group work:** Even though the project is completed as a group, **every student must upload the final version of the project individually.**
- **Presentation:** Project presentations will be held during the **laboratory session on December 25, 2026**.
- **Grading Policy:**
    - The final grade is calculated as follows:
        - 50% : Code and submission
        - 50% : Project presentation
    - **Students who do not attend the presentation will receive a grade of zero for the entire project**